Honeypot-Based Threat Detection System: Capturing and Analyzing Malicious Actor Behavior

Rei Thao
School of Engineering and Computer Science
of University of the Pacific
Stockton, California
r_thao1@u.pacific.edu

Diego Arroyo
School of Engineering and Computer Science
of University of the Pacific
Stockton, California
d_arroyo5@u.pacific.edu

***Abstract*—Modern organizations face an escalating volume of cyberattacks that traditional perimeter-based and reactive security tools are ill-equipped to fully address. Intrusion detection systems and firewalls can identify and block known threats, but they provide limited insight into attacker methodology, tooling, and behavior once a system is compromised. This gap in defensive intelligence makes it difficult for security teams to anticipate attack patterns, improve detection logic, and build more resilient defenses. Honeypots address this limitation by deploying intentionally vulnerable decoy systems that lure malicious actors into a monitored environment, enabling defenders to observe attacker behavior without risk to production infrastructure.**

Despite their utility, honeypot deployments in educational and small enterprise contexts are often limited to single-service configurations that capture only narrow attack vectors. Multi-layer honeypot frameworks capable of simultaneously capturing SSH brute force activity, malware delivery attempts, and credential theft across unified logging and alerting pipelines remain underexplored in applied academic settings. Additionally, integrating real-time alerting and visual analytics into such deployments is rarely demonstrated end-to-end in a validated, reproducible environment.

This project addresses these gaps by designing, deploying, and validating a multi-layer honeypot-based threat detection system using the T-Pot CE framework on Ubuntu 24.04 in an isolated VMware environment. The system integrates Cowrie for SSH and Telnet deception, Dionaea for malware capture, and Canarytokens for credential-use alerting, unified under an Elasticsearch and Kibana stack for log aggregation and visualization. A custom Python alerting pipeline provides real-time email notification upon attacker session activity. The detection pipeline was validated through a structured five-stage simulated attack conducted from a Kali Linux environment using Nmap, Hydra, and standard post-exploitation techniques. Results confirmed successful detection across all attack stages, demonstrating the viability of multi-layer honeypot deployments as a practical and accessible mechanism for generating actionable defensive threat intelligence.

Keywords—Canarytokens, cyber deception, Cowrie, Dionaea, honeypot, intrusion detection, MITRE ATT&CK, network security, SSH honeypot, T-Pot CE, threat intelligence

I. Introduction

Modern organizations face an increasing volume and sophistication of cyberattacks. Traditional perimeter defenses such as firewalls and intrusion detection systems are reactive by nature; they detect and block known threats but provide limited visibility into attacker behavior, methods, and motivations. Honeypots offer a complementary approach: by deploying decoy systems that simulate vulnerable services, security teams can observe attackers in a controlled environment without exposing production infrastructure to risk.

The goal of this project is to design, deploy, and validate a multi-layer honeypot system that captures attacker behavior across multiple attack vectors. The system must be configurable, realistic enough to deceive automated scanners and manual attackers, and capable of generating actionable intelligence through logging, visualization, and real-time alerting.

This paper is organized as follows: Section II describes system architecture. Section III details methodology. Section IV presents results. Section V discusses challenges encountered. Section VI provides the deployment procedure. Section VII covers conclusions and future work.

II. System Architecture

A. Component Overview

The system is built on T-Pot CE (Community Edition), a multi-honeypot platform developed by Telekom Security (Deutsche Telekom). T-Pot CE orchestrates multiple honeypot services as Docker containers on a single host, with a centralized Elasticsearch/Kibana stack for log aggregation and visualization. The following components were deployed:

Cowrie — SSH/Telnet honeypot emulating a Linux shell, capturing login attempts, commands, and file transfer activity.

Dionaea — Low-interaction honeypot emulating vulnerable services (SMB port 445, FTP port 21, HTTP port 80, MSSQL port 1433) to capture malware samples.

Canarytokens — Web-based tokens that trigger email alerts when fake credentials or documents are accessed by an attacker.

Suricata — Network IDS/IPS inspecting traffic against known CVE signatures.

Kibana/Elasticsearch — Visualization and storage backend for all honeypot logs, providing dashboards for attack maps and session statistics.

B. Infrastructure

The system runs on a dedicated laptop with 32GB RAM and an Intel i5 processor, running Ubuntu 24.04 LTS within a

VMware virtual machine configured in NAT mode. Ports 22 and 23 are forwarded to the Cowrie container. Real administrative SSH access to the host uses port 64295. All honeypot services run as Docker containers managed by Docker Compose, with configuration files and log outputs persisted via volume mounts to the host filesystem.

C. Docker Volume Architecture

Four Docker volume mounts connect the host filesystem to the Cowrie container, enabling persistent customization across container restarts and image updates. An additional volume mount was required to override the baked-in cowrie.cfg at the container root, as Cowrie loads configuration files in a specific priority order that caused the mounted configuration to be overridden by internal defaults.

Host Path (data/cowrie/…)	**Container Path (/home/cowrie /cowrie/…)**	**Purpose**
keys/	etc/	Config, userdb, pickle
downloads/	dl/	Captured malware
log/	log/	JSON/text logs
tty/	log/tty/	TTY session recordings

Table I. Docker Volume Mounts for Cowrie

III. Methodology

A. Phase 1 — Infrastructure Setup

T-Pot CE was installed on Ubuntu 24.04, on a device containing 32Gb of RAM and an i5 Intel processor, following the official procedure from the Telekom Security GitHub repository. The TPOT_DATA_PATH environment variable in the .env configuration file was set to an absolute path, as the default relative path (./data) caused all four Cowrie volume mounts to resolve to the same directory.

B. Phase 2 — Cowrie Configuration

Cowrie required significant configuration to create a realistic honeypot environment across four sub-components:

1) User Authentication: A userdb.txt file was created defining fake accounts in the format username:uid:password. The auth_class was changed from the default AuthRandom (which randomly accepts any credential) to UserDB, enforcing the defined credential list.

2) Fake Filesystem: Cowrie uses a Python pickle file (fs.pickle) to store a virtual filesystem tree. The 10-element list format was identified by reading Cowrie source code (shell/fs.py): [name, type, uid, gid, size, mode, timestamp, children, target, realfile]. Type constants T_FILE=2 and T_DIR=1 were critical; early attempts used arbitrary inode numbers in position [1], causing init_honeyfs() to skip all custom entries.

3) HoneyFS Contents: Cowrie maps virtual paths to real file contents via a honeyfs directory, populated at startup by init_honeyfs(). The volume mount was adjusted to match the contents_path = etc/honeyfs configuration setting.

4) /etc/passwd Integration: A critical discovery was that Cowrie reads honeyfs/etc/passwd to determine user home directories via the Passwd class in pwd.py. Fake users were added to this file in standard Unix format to enable correct home directory resolution.

C. Fake User Environment Design

Four fake user accounts were created to simulate a realistic small company server environment. Each user has a distinct role, set of files, and relationships to other users that create a believable narrative for attackers to follow. Table II summarizes the user roles and their associated deception artifacts.

User	**Role**	**Key Files**	**Canary-token**
root	System Admin	server_con -fig.txt, .bash_history	None
admin	IT Administra -tor	passwords. txt, aws_credentia -ls.txt, database_back -up.sql, server_notes.t-xt	AWS Key Token, Word Document Token
rei	Develo-per	db_config. py, app.py, notes.txt, requirements.t -xt	Web Bug URL in notes.txt
mack	Intern	onboardin-g.txt, todo.txt	Web Bug URL in onboarding.tx -t

D. Phase 3 — Canarytoken Integration

Three Canarytoken types were generated via canarytokens.org:

AWS Key Token embedded in admin/aws_credentials.txt — triggers alert on programmatic AWS API access using the fake credentials.

Microsoft Word Token placed as admin/confidential_report.docx — triggers when the document is opened in Microsoft Office.

Web Bug URLs embedded in rei/notes.txt and mack/onboarding.txt — trigger on any HTTP request to the embedded URL.

E. Phase 4 — Email Alerting

A Python script monitors Cowrie's JSON log using a tail approach (seeking to end of file and polling for new lines). The script tracks sessions by ID, accumulating commands. On session close (cowrie.session.closed event), it dispatches a summary email via Gmail SMTP containing the attacker's username, source IP, all commands executed, and the exact TTY playback command for forensic replay.

F. Phase 5 — Attack Simulation

Attack simulations were conducted from a Kali Linux VM on the same isolated network. Table II maps each stage to the MITRE ATT&CK framework.

#	MITRE Tactic	Technique	Tool/Action
1	Reconnaissance	T1595	Nmap -sV
2	Initial Access	T1110 Brute Force	Hydra SSH
3	Discovery	T1033, T1087	whoami, cat /etc/passwd
4	Credential Access	T1552	cat passwords.txt
5	Lateral Movement	T1021	SSH mack→rei→admin
6	Exfiltration	T1048	wget, curl
7	Command & Control	T1071	Canarytoken trigger

Table II. MITRE ATT&CK Attack Stage Mapping

IV. Results

A. Attack Detection Statistics

Over a 24-hour monitoring window, the system captured the statistics in Table III.

Metric	Value
Total attacks detected	352
Cowrie SSH/Telnet attempts	323
Dionaea malware capture attempts	27
Tanner web honeypot hits	2
CVE-2024-6387 probes (Suricata)	51
Unique attacker source IPs	2 (isolated lab)
Email alerts triggered	Multiple per session

Table III. 24-Hour Attack Detection Summary

B. Credential Analysis

Kibana credential tagclouds revealed the most frequently targeted usernames (admin, root, rei, mack) and passwords (admin, 123456, password, reithao, mack123). The system captured all simulated Hydra brute force attempts, recording each username/password combination in the Cowrie JSON log. Comparing attempted credentials against planted fake credentials demonstrates the honeypot's ability to identify targeted credential lists.

C. Session Playback

Cowrie recorded full TTY session logs for all interactive SSH sessions, replayable using the built-in playlog utility. This provides forensic value beyond simple command logging by capturing timing, navigation patterns, and erroneous keystrokes, enabling detailed post-incident analysis.

D. Canarytoken Alerting

All three Canarytoken types generated email alerts within seconds of being triggered during simulated sessions. The AWS credential token, Word document token, and Web Bug URL tokens each functioned as designed, demonstrating multi-layer deception capability that alerts defenders when attackers interact with planted artifacts.

E. CVE Detection

Suricata detected 51 instances of CVE-2024-6387 (OpenSSH regreSSHion) probing against the honeypot, demonstrating that the IDS component provides meaningful signature-based detection complementing the behavioral capture of Cowrie and Dionaea.

V. Challenges and Solutions

A. Docker Volume Resolution

The TPOT_DATA_PATH relative path caused all four Cowrie volume mounts to resolve identically to the downloads directory. Diagnosed by inspecting docker inspect output and resolved by setting an absolute path in the .env configuration file.

B. Configuration Loading Order

Cowrie loads four configuration files in priority order, with the root cowrie.cfg taking precedence. The mounted etc/cowrie.cfg was being overridden by the container's internal default. Resolved by adding a volume mount that directly overlays the root cowrie.cfg path.

C. Pickle Format

The fs.pickle format was undocumented. Source code analysis of shell/fs.py identified the 10-element list structure and type constants. Incorrect use of inode numbers in the type field (position [1]) caused init_honeyfs() to skip all custom filesystem entries during the A_REALFILE update pass.

D. HoneyFS Path Mismatch

Files appeared in ls output, but cat returned "No such file or directory." The volume mount pointed to /honeyfs but

contents_path expected /etc/honeyfs — different container paths. Correcting the mount destination resolved the mismatch.

E. Home Directory Resolution

The cat command failed despite correct filesystem and honeyfs configuration. Tracing through avatar.py revealed that home directories are sourced from honeyfs/etc/passwd via the Passwd class in pwd.py. Adding fake users to honeyfs/etc/passwd was the final required fix.

VI. Deployment and Reproduction

A. Using the Pre-Configured VM (Recommended)

The submission includes a compressed Ubuntu 24.04 VMware virtual machine with T-Pot CE fully installed and configured. This is the recommended method for reproducing the results described in this paper.

1) System Requirements:

- VMware Workstation, VMware Player, or VirtualBox
- Host machine with minimum 8GB RAM (16GB recommended for the VM)
- 50GB free disk space for the VM image
- Host-only or NAT network adapter

2) Import and Start:

```
1. Extract the .zip archive
2. Open VMware → File → Open → select the .vmx file
3. Set network adapter to NAT
4. Power on the VM
5. Login: username: drhp, password: drhp
```

3) Verify Services Are Running:

```
systemctl status tpot
docker ps | grep -E "cowrie|dionaea"
```

4) 4) Start the Email Alert Script:

```
nohup python3 ~/alert.py > ~/alert.log 2>&1 &
```

Update EMAIL_FROM, EMAIL_TO, and APP_PASSWORD in ~/alert.py with your Gmail credentials and a valid Gmail app password before running, otherwise all SSH attempts captured through Cowrie will be sent to a project owner's email.

5) Access the Dashboard:

- Kibana dashboard: https://<vm-ip>:64297
- Admin SSH access: ssh drhp@<vm-ip> -p 64295
- Honeypot SSH (Cowrie): ssh <user>@<vm-ip> -p 22

6) Find the VM IP Address:

```
ip addr show | grep inet
```

7) View TTY Session Playback:

Cowrie records full TTY session logs of every attacker interaction. To replay a session, first list available recordings:

```
ls ~/tpotce/data/cowrie/log/tty/
```

To find the most recent session (newest first):

```
ls -lt ~/tpotce/data/cowrie/log/tty/
```

Play back a specific session using Cowrie's built-in playlog utility:

```
docker exec -it cowrie python3 bin/playlog \

/home/cowrie/cowrie/log/tty/<session-id>
```

Replace <session-id> with the full filename from the tty directory. The playback replays every keystroke the attacker made in real time, including timing between commands. The exact playback command is also included in the email alert sent when a session closes

B. Fresh Installation (Alternative)

For those who prefer to deploy from scratch rather than using the provided VM image:

1) Prerequisites:

- Ubuntu 24.04 LTS with minimum 8GB RAM (32GB recommended)
- Docker and Docker Compose installed
- VMware or VirtualBox for network isolation

2) T-Pot CE Installation:

```
git clone https://github.com/telekom-security/tpotce
cd ~/tpotce && sudo ./install.sh
```

Set absolute data path in .env:

```
TPOT_DATA_PATH=/home/<user>/tpotce/data
```

3) Cowrie Configuration:

Create cowrie.cfg in the keys directory with the following settings:

```
[honeypot]
auth_class = UserDB
contents_path = etc/honeyfs
[shell]
filesystem = etc/fs.pickle
userdb = etc/userdb.txt
[output_jsonlog]
enabled = true
logfile = log/cowrie.json
```

Add volume mount in docker-compose.yml to override root config:

```
- keys/cowrie.cfg:/home/cowrie/cowrie/cowrie.cfg:ro
```

4) Fake Environment Setup:

Create userdb.txt with fake user credentials and honeyfs/etc/passwd with matching user entries. Obtain a clean fs.pickle from the container image via docker cp, run addfiles.py to add fake user directories and files, then add the pickle volume mount. Create fake file content in the honeyfs directory structure. See Appendix A for full code.

5) Start System and Alerting:

```
systemctl start tpot
nohup python3 ~/alert.py > ~/alert.log 2>&1 &
```

Add @reboot entry to crontab for persistence across reboots.

C. File Structure Reference

Key files included in the submission:

```
~/tpotce/data/cowrie/keys/
├── cowrie.cfg                # Cowrie configuration
├── userdb.txt                # Fake user credentials
├── fs.pickle                 # Fake filesystem
└── honeyfs/                  # Fake file contents
    ├── etc/passwd            # Fake user entries
    └── home/
        ├── root/             # root(System Administrator) fake files
        ├── admin/            # admin(IT Administrator) fake files
        ├── rei/              # rei (developer) fake files
        └── mack/             # mack (Intern) fake files
~/alert.py                    # Email alerting script
~/addfiles.py                 # Filesystem builder script
```

VII. Conclusion and Future Work

A. Conclusion

This project successfully deployed a multi-layer honeypot system demonstrating effective capture of attacker behavior across SSH brute force, malware upload, and credential theft scenarios. The MITRE ATT&CK-mapped simulation validated detection across seven attack tactics. Non-trivial engineering challenges including Docker volume debugging, Cowrie source code analysis, and pickle format reverse engineering were solved, providing hands-on experience beyond theoretical instruction. The complete system was built using exclusively open-source tools at zero software licensing cost.

B. Future Work

- Public internet deployment for real-world attack data collection, currently limited by university NAT environment.
- Expanded Canarytoken coverage including DNS tokens embedded as fake internal server hostnames.
- SIEM integration for automated threat correlation and incident response workflows.
- Additional honeypot types, including ICS/SCADA simulation and web application honeypots.
- Automated attacker behavior profiling and clustering analysis from captured sessions.

Acknowledgements

We thank Shawn Kerns and Dr. Pramod Gupta for their guidance and support throughout this project. The authors also acknowledge the T-Pot CE project by Telekom Security (Deutsche Telekom) for the open-source honeypot framework, and the canarytokens.org project for credential alerting infrastructure.

References

[1] Telekom Security, “T-Pot CE: A Multi-Honeypot Platform,” GitHub, 2024. [Online]. Available: https://github.com/telekom-security/tpotce

[2] Thinkst Applied Research, "Canarytokens," 2024. [Online]. Available: https://canarytokens.org

[3] MITRE Corporation, "MITRE ATT&CK Framework," 2024. [Online]. Available: https://attack.mitre.org